

Lecture 01 Handout — The Note-Set Starter: Two Specifications and the Working Rules

Software-engineering module, Lecture 01. Read the two specifications before the first meeting (five minutes). Keep this handout open during the demo and while doing Exercise 1.

This handout reproduces the three files that the Lecture 01 demo repository contains at the start of class, before any note exists. They are copied from `demos/lecture-01-note-specs-demo/starter/` in the course repository; only the heading levels are shifted so that the handout has one title.

File	Role in the lecture
<code>note-set-conops.md</code>	The concept of operations: what the note set is for, who uses it, and the operations the agent performs. A specification of purpose and behavior.
<code>note-format-spec.md</code>	The format specification: six numbered rules, R1 through R6, that every note must satisfy. The primary example of a specification in the lecture.
<code>CLAUDE.md</code>	The working rules for the agent: read the specifications first, never change one without approval, check conformance after every operation, report before repairing.

Both specifications are version 0.1 and are marked as drafts. They contain defects. During the demo the agent is asked to find them before planning, and the format specification is amended to version 1.0.0 (a scope statement, an extended R3, a new R7) and later to 2.0.0 (a required Summary section, R8, and an amended R6). The versions in this handout are the starting point, not the end state; the end state is in `demos/lecture-01-note-specs-demo/completed/`.

Exercise 1 starts from a copy of these three files.

File: `starter/note-set-conops.md` — The concept of operations.

Note Set — Concept of Operations

Version: 0.1 (draft) Status: draft for review; not yet realized.

1. Purpose and scope

A single user keeps a personal set of study notes on software-engineering classics. A coding agent performs all structural operations on the note set; the user writes and edits note content. This document describes the note set and the operations the agent supports, from the user's point of view.

2. The note set

Notes are markdown files stored in a `notes/` directory inside a git repository. An index file, `index.md`, at the top level of the repository links to every note in the set. The formatting of markdown files in the note set is governed by `note-format-spec.md`.

3. Roles

There is one user and one agent (Claude Code). The user requests operations conversationally; the agent performs them and reports what changed.

4. Agent operations

O1 — Add blank note

- **Input:** a title.
- **Precondition:** no note with that title exists in the set.
- **Postcondition:** a new note exists containing well-formed front-matter and the title as its heading, with no other content; `index.md` lists it.

O2 — Add note with initial content

- **Inputs:** a title and initial markdown text.
- **Precondition:** no note with that title exists in the set.
- **Postcondition:** a note exists whose content is the supplied text brought into conformance with `note-format-spec.md`, preserving the user's meaning; `index.md` lists it.

O3 — Remove note

- **Input:** a title.
- **Precondition:** a note with that title exists in the set.
- **Postcondition:** the note file no longer exists; `index.md` no longer lists it.

5. Conformance

Every note in the set is expected to conform to `note-format-spec.md` at all times — conformance is an invariant of the set, not a milestone. The user may at any time direct the agent to check one note, or all notes, for conformance. A conformance report cites the specific rule identifiers (R1, R2, ...) from the format spec.

6. Change management

Every operation ends in a git commit. The specifications themselves live in the same repository and are maintained documents: when they change, they change by versioned, reviewed edits.

File: `starter/note-format-spec.md` — The format specification.

Note Format Specification

Version: 0.1 (draft) Status: draft for review; not yet realized.

This is a normative specification. The keywords **MUST**, **MUST NOT**, **SHOULD**, and **MAY** carry their RFC-2119 meanings. Rules are numbered so that conformance reports — whether produced by an agent or by a tool — can cite them. Every markdown file in the note set **MUST** satisfy the rules below.

Rules

R1 — Front-matter block Every note **MUST** begin with a YAML front-matter block delimited by `---` lines. The opening `---` **MUST** be the first line of the file, and a closing `---` **MUST** follow the front-matter fields.

R2 — Front-matter fields The front-matter **MUST** contain exactly two fields:

- `title` — a non-empty string.
- `created` — the note's creation date in ISO-8601 form (YYYY-MM-DD).

R3 — Title heading The note body **MUST** begin with a level-1 heading (`#`), which **MUST** be the first non-blank line after the front-matter.

R4 — Single H1 A note MUST contain exactly one level-1 heading.

R5 — No skipped heading levels A heading MUST be at most one level deeper than the nearest heading above it. A `##` section MAY open `###` subsections, but a `##` heading MUST NOT be followed directly by a `####` heading.

R6 — Index consistency Every note file MUST be linked from `index.md` exactly once, and every note link in `index.md` MUST resolve to an existing note file.

File: `starter/CLAUDE.md` — The working rules for the agent.

Note set — working rules for the agent

An agent-maintained set of markdown study notes. Governing documents: `note-set-conops.md` (what the system is for) and `note-format-spec.md` (what a conformant note is).

Working rules

1. Read `note-set-conops.md` and `note-format-spec.md` in full before performing any note operation.
2. Never modify `note-set-conops.md` or `note-format-spec.md` without explicit user approval. Propose spec changes as a numbered list of amendments and wait.
3. After any operation that creates, changes, or removes a note, check the affected notes for conformance and report the result — citing rule IDs — before declaring the operation complete.
4. When a conformance check finds violations, report them first and wait for the user's go-ahead before repairing anything.
5. When reporting a violation, quote the violated rule's text from `note-format-spec.md` alongside the finding.
6. Keep `index.md` consistent with the contents of `notes/` at all times (rule R6). Never edit `index.md` except as part of an operation.
7. Once `note-set-operations.md` exists, perform operations exactly as documented there; if its documentation is wrong or incomplete, propose an update rather than improvising.
8. End every completed operation with a git commit whose message names the operation (for example, `add-blank-note: The Mythical Man-Month`).